

Test Technique — Python

Algorithmique & Manipulation de Structures de Données

DURÉE ESTIMÉE

20 à 30 minutes

ENVIRONNEMENT

Python 3.10+ (Standard Library)

LIVRABLE

Fonction unique complète

1. OBJECTIF

Écrire une fonction Python `calculate_total(cart: list[dict], rules: dict) -> float` qui calcule le total d'un panier d'articles après application de règles de remise complexes.

2. RÈGLES MÉTIER

- **Règle 1 — Remise au Volume (par article)** : Si la quantité commandée d'un même article atteint un seuil défini dans `rules["volume_discounts"]`, son prix unitaire bénéficie de la remise associée (ex: -10% pour ≥ 5 articles).
- **Règle 2 — Offre Groupée "Pack 3 pour 2" (par catégorie)** : Pour les catégories éligibles répertoriées dans `rules["bundle_categories"]`, pour chaque tranche de 3 articles achevés (tous produits de la catégorie confondus), l'article le moins cher des 3 est **offert** (0€).

⚠ Ordre d'Application & Précisions

La remise au volume (Règle 1) s'applique **avant** le calcul de la gratuité (Règle 2). La gratuité s'applique sur le prix unitaire déjà remis. Le total final doit être un `float` arrondi à 2 décimales.

3. EXEMPLE D'EXÉCUTION & SQUELETTE

Votre code doit valider l'exemple suivant :

```
def calculate_total(cart: list[dict], rules: dict) -> float:
    # TODO: Implémenter la logique
    pass

# DONNÉES DE TEST
rules = {
    "volume_discounts": {5: 0.10, 10: 0.20}, # qty >= 5: -10%, qty >= 10: -20%
    "bundle_categories": ["book"]             # 3e offert par tranche de 3
}

cart = [
    {"id": "A", "category": "book", "price": 10.0, "qty": 4},
    {"id": "B", "category": "book", "price": 15.0, "qty": 1},
    {"id": "C", "category": "tech", "price": 100.0, "qty": 1},
]

# Calcul attendu :
# - 'book': 5 livres au total (4x A à 10€ + 1x B à 15€). Pas de remise volume (aucun article >= 5).
# - Offre Pack 'book': 5 livres = 1 groupe de 3 + 2 restants. Le moins cher du groupe (un livre 'A' à 10€) est gratuit.
# - Total 'book' : (3 x 10€) + (1 x 15€) = 45.00€
# - Total 'tech' : 1 x 100€ = 100.00€
# TOTAL FINAL EXPECTED = 145.00

assert calculate_total(cart, rules) == 145.00
```